MLFP mlfp06 — Textbook Chapter

- Module 6 — Machine Learning with Language Models and Agentic Workflows
  - Learning Outcomes
  - Prerequisites
  - How to Read This Chapter
- Lesson 6.1: LLM Fundamentals, Prompt Engineering, and Structured Output
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: How LLMs are trained
    - FOUNDATIONS: Prompt engineering techniques
    - FOUNDATIONS: Kaizen structured output
    - ADVANCED: Inference considerations
  - Mathematical Foundations
    - THEORY: The softmax temperature
  - The Kailash Engine: Kaizen Delegate
  - Worked Example: Prompt Engineering Comparison
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 6.2: LLM Fine-Tuning — LoRA, Adapters, and the Technique Landscape
  - Why This Matters
  - Core Concepts
    - THEORY: LoRA — Low-Rank Adaptation
    - THEORY: Adapter layers
    - FOUNDATIONS: LoRA vs Adapter comparison
    - FOUNDATIONS: The fine-tuning landscape (survey)
    - ADVANCED: Model merging
    - FOUNDATIONS: Quantisation
  - Mathematical Foundations
    - THEORY: Why low rank works
  - The Kailash Engine: kailash-align
  - Worked Example: LoRA Fine-Tuning for Sentiment Classification
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 6.3: Preference Alignment — DPO and GRPO
  - Why This Matters
  - Core Concepts
    - THEORY: From RLHF to DPO — the derivation
    - THEORY: The Bradley-Terry preference model
    - THEORY: The DPO loss
    - THEORY: GRPO — Group Relative Policy Optimization
    - FOUNDATIONS: LLM-as-Judge
    - FOUNDATIONS: Evaluation benchmarks
  - The Kailash Engine: kailash-align (DPO)
  - Worked Example: DPO Training on Preference Data
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 6.4: RAG Systems
  - Why This Matters

- Core Concepts
  - FOUNDATIONS: The RAG pipeline
  - THEORY: BM25 — sparse retrieval
  - THEORY: Cosine similarity — dense retrieval
  - FOUNDATIONS: Hybrid retrieval
  - FOUNDATIONS: RAGAS evaluation
  - ADVANCED: HyDE (Hypothetical Document Embeddings)
- Worked Example: RAG on Singapore Policy Documents
- Try It Yourself
- Cross-References
- Reflection

- Lesson 6.5: AI Agents — ReAct, Tool Use, and Function Calling
  - Why This Matters
  - Core Concepts
    - THEORY: ReAct formalisation
    - FOUNDATIONS: Function calling
    - FOUNDATIONS: Cost budget safety
    - FOUNDATIONS: Agent design framework
  - Worked Example: Data Analysis Agent with ReAct
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 6.6: Multi-Agent Orchestration and MCP
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: Multi-agent patterns
    - FOUNDATIONS: MCP (Model Context Protocol)
    - FOUNDATIONS: Agent memory
  - Worked Example: Multi-Agent ML Pipeline
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 6.7: AI Governance Engineering
  - Why This Matters
  - Core Concepts
    - THEORY: PACT D/T/R addressing
    - FOUNDATIONS: Operating envelopes
    - FOUNDATIONS: Budget cascading
    - FOUNDATIONS: Governance testing
    - FOUNDATIONS: PactGovernedAgent
  - Worked Example: Governed Multi-Agent System
  - Try It Yourself
  - Cross-References
  - Reflection
- Lesson 6.8: Capstone — Full Production Platform
  - Why This Matters
  - Core Concepts
    - FOUNDATIONS: Nexus multi-channel deployment
    - FOUNDATIONS: Authentication and authorisation
    - FOUNDATIONS: Full platform integration
    - FOUNDATIONS: Debugging agent reasoning
  - Worked Example: Deploying the M6 System
  - Try It Yourself
  - Cross-References

    - Reflection
- Chapter Summary
    - The complete MLFP arc
- Glossary
- Further Reading

Module 6 — Machine Learning with Language Models and Agentic Workflows

> *"The model is not the product. The governed system is the product."*

This is the final chapter of the MLFP programme. You arrived here from a long road: Python basics and data wrangling in Module 1, statistics and probability in Module 2, the full supervised ML pipeline in Module 3, unsupervised learning and the neural network bridge in Module 4, and every major deep learning architecture in Module 5. Everything you have learned converges here.

Module 6 is about building production AI systems with large language models. Not using them — building them. You will engineer prompts, fine-tune models with LoRA, align them with human preferences using DPO, ground them in facts with RAG, give them tools through agents, coordinate multiple agents, govern them with PACT, and deploy them with Nexus. Every lesson produces running code. Every system you build is governed — with access controls, cost budgets, operating envelopes, and audit trails.

The organising principle of Module 6 is the transition from model to system. A model takes an input and produces an output. A system takes a goal and achieves it — reasoning about sub-tasks, selecting tools, retrieving knowledge, coordinating specialists, respecting boundaries, and explaining its decisions. The distance between a model and a system is the distance between a trained neural network and a production application. This chapter covers that distance.

By the end of this chapter you will have deployed a complete, governed, multi-agent AI system accessible via API, CLI, and MCP simultaneously. That is the capstone of the MLFP programme, and it is the starting point of your career as an ML engineer.

Learning Outcomes

By the end of this chapter you will be able to:

- Use LLMs effectively with prompt engineering techniques (zero-shot, few-shot, chain-of-thought, self-consistency) and extract structured output using Kaizen's Delegate and Signature APIs.
- Implement LoRA from scratch, understanding the low-rank factorisation mathematics, and implement adapter layers from scratch. Survey all 10+ fine-tuning techniques and select the right one for a given scenario.
- Derive the DPO loss function from the Bradley-Terry preference model and the RLHF objective. Implement DPO training with preference pairs. Explain GRPO and when to prefer it over DPO. Evaluate aligned models with LLM-as-judge and standard benchmarks.
- Build complete RAG pipelines with chunking, dense retrieval, sparse retrieval (BM25), hybrid retrieval, re-ranking, and HyDE. Evaluate RAG quality with RAGAS metrics.
- Build ReAct agents with custom tools, implement function calling with structured schemas, and enforce cost budgets to prevent runaway spending.
- Implement multi-agent patterns (supervisor-worker, sequential, parallel, handoff), build MCP servers, and configure agent memory.

- Implement PACT governance with D/T/R addressing, operating envelopes, budget cascading, and governance testing.
- Deploy a complete AI system with Nexus (API + CLI + MCP), implement authentication, integrate drift monitoring, and verify governance at the deployment level.

Prerequisites

Module 5 complete. This chapter assumes you can:

- Fine-tune pre-trained models (BERT, ResNet) using transfer learning.
- Implement and train any major deep learning architecture.
- Export models for production with ONNX.
- Explain how PPO is used in RLHF for LLM alignment.

From Module 4 specifically: TF-IDF and BM25 (Lesson 4.6) and word embeddings (Lesson 4.6) — these reappear in the RAG lesson.

From Module 3 specifically: Evaluation metrics and drift monitoring (Lessons 3.5, 3.8) — these reappear in the capstone.

Notation:

- π_θ is a policy (the LLM) parameterised by θ.
- π_{ref} is a reference policy (the original, unaligned model).
- y_w and y_l are preferred (winning) and dispreferred (losing) responses.
- β is the KL-penalty coefficient in DPO.
- σ is the sigmoid function.

How to Read This Chapter

Same structure as all previous modules. The scaffolding level is minimal (~20% code provided) — you are now a fluent ML engineer.

Estimated reading time per lesson:

Lesson	Title	Reading	Exercise	Total
6.1	LLM Fundamentals and Prompt Engineering	100 min	60 min	~2h 40m
6.2	LLM Fine-Tuning — LoRA, Adapters, and the Technique Landscape	130 min	80 min	~3h 30m
6.3	Preference Alignment — DPO and GRPO	120 min	75 min	~3h 15m
6.4	RAG Systems	110 min	70 min	~3h
6.5	AI Agents — ReAct, Tool Use, and Function Calling	110 min	65 min	~2h 55m

Lesson	Title	Reading	Exercise	Total
6.6	Multi-Agent Orchestration and MCP	110 min	70 min	~3h
6.7	AI Governance Engineering	100 min	65 min	~2h 45m
6.8	Capstone — Full Production Platform	120 min	80 min	~3h 20m

Total: roughly 24 hours.

Lesson 6.1: LLM Fundamentals, Prompt Engineering, and Structured Output

Why This Matters

The transformer architecture you built in Lesson 5.4 is the foundation of every large language model. GPT, Claude, Gemini, Llama — they are all transformers, scaled to billions of parameters and trained on trillions of tokens. But a pre-trained language model is not a product. It is a foundation. To turn it into a product, you need three capabilities: the ability to control its output (prompt engineering), the ability to customise it for your domain (fine-tuning, Lesson 6.2), and the ability to align it with human values (preference alignment, Lesson 6.3).

This lesson focuses on the first capability: prompt engineering. The difference between a well-prompted and poorly-prompted LLM can be the difference between a useful system and an unreliable one. You will learn five prompting techniques, from zero-shot to self-consistency, and you will use Kaizen's structured output APIs to extract type-safe results instead of free-form text.

Core Concepts

FOUNDATIONS: How LLMs are trained

An LLM like GPT is trained in two stages:

Pre-training. The model learns to predict the next token in a sequence. Given "The capital of Singapore is", the model learns to assign high probability to "Singapore" (or rather, to the token that represents "Singapore"). The training corpus is a large fraction of the internet — books, Wikipedia, code, web pages. Pre-training on trillions of tokens gives the model a broad understanding of language, facts, reasoning patterns, and code.

Alignment. The pre-trained model is a next-token predictor, not a helpful assistant. It will happily complete a harmful prompt or generate nonsense that looks authoritative. Alignment tunes the model to be helpful, harmless, and honest. This is done through RLHF (Reinforcement Learning from Human Feedback) or DPO (Direct Preference Optimization, Lesson 6.3): human annotators rank model outputs, and the model is trained to prefer the higher-ranked outputs.

Scaling laws. Model performance scales predictably with three factors: the number of parameters, the amount of training data, and the amount of compute. Doubling any one of these produces a predictable improvement in loss. This is why LLMs have grown from millions of parameters (GPT-1, 2018) to hundreds of billions (GPT-4, 2023).

FOUNDATIONS: Prompt engineering techniques

Zero-shot prompting. Give the model only a task description, no examples:

```
Classify the following review as positive or negative:
Review: "The laksa at this hawker stall is the best I've had in Katong."
Sentiment:
```

Few-shot prompting. Provide examples before the query:

```
Review: "Excellent char kway teow, generous portions." -> Positive
Review: "Too salty, overpriced for hawker standards." -> Negative
Review: "The laksa at this hawker stall is the best I've had in Katong." ->
```

Chain-of-thought (CoT). Prompt the model to reason step by step:

```
Q: A Singapore taxi charges S$3.90 flag-down + S$0.25 per 400m.
   What is the fare for a 12km trip?

Let's think step by step:
1. Total distance: 12,000m
2. Number of 400m units: 12,000 / 400 = 30
3. Distance charge: 30 × S$0.25 = S$7.50
4. Total fare: S$3.90 + S$7.50 = S$11.40
```

Zero-shot CoT. Append "Let's think step by step" without providing examples. Surprisingly effective — it activates the model's reasoning capabilities without requiring hand-crafted chain-of-thought examples.

Self-consistency. Sample multiple chain-of-thought paths and take the majority vote. This reduces the variance of CoT prompting by aggregating diverse reasoning paths.

FOUNDATIONS: Kaizen structured output

Free-form text is unreliable for production systems — the output format varies between calls. Kaizen provides structured output through Signatures:

```python
from kailash_kaizen import Delegate, Signature, InputField, OutputField

class SentimentSignature(Signature):
    """Classify the sentiment of a product review."""
    review: str = InputField(description="The review text")
    sentiment: str = OutputField(description="One of: positive, negative, neutral")
    confidence: float = OutputField(description="Confidence score 0-1")

delegate = Delegate(model="claude-3-haiku")
result = delegate.run(SentimentSignature, review="Best laksa in Katong!")
print(result.sentiment)    # "positive"
print(result.confidence)   # 0.95
```

The Signature defines the input and output schema. The Delegate handles the LLM call, streaming, cost tracking, and structured parsing. This is type-safe — you get Python objects, not strings.

ADVANCED: Inference considerations

KV-cache. During autoregressive generation, the model recomputes attention over all previous tokens at each step. The KV-cache stores the key and value matrices from previous steps, avoiding redundant computation. This is why LLM inference is memory-bound, not compute-bound.

Speculative decoding. A small, fast "draft" model generates candidate tokens, which a larger model verifies in parallel. This can speed up generation by 2–3× without changing the output distribution.

Continuous batching. Instead of waiting for all requests in a batch to finish, new requests are added to the batch as old ones complete. This maximises GPU utilisation for serving.

Mathematical Foundations

THEORY: The softmax temperature

The LLM's output is a probability distribution over the vocabulary, computed via softmax with a temperature parameter T:

$$P(w_i) = \frac{e^{z_i / T}}{\sum_j e^{z_j / T}}$$

At $T = 1$ (default), the distribution is as trained. At $T < 1$, the distribution becomes peakier (more deterministic — the model is more confident). At $T > 1$, the distribution becomes flatter (more random — the model explores more). Temperature 0 is equivalent to argmax (always pick the most likely token). For factual tasks, use low temperature; for creative tasks, use higher temperature.

The Kailash Engine: Kaizen Delegate

```
from kailash_kaizen import Delegate, LLMCostTracker

tracker = LLMCostTracker(budget_usd=1.00)
delegate = Delegate(model="claude-3-haiku", cost_tracker=tracker)

# Track costs across multiple calls
for review in reviews:
    result = delegate.run(SentimentSignature, review=review)
    print(f"Cost so far: ${tracker.total_cost:.4f}")
```

Worked Example: Prompt Engineering Comparison

```
from kailash_kaizen import Delegate, Signature, InputField, OutputField

class MathSolver(Signature):
    """Solve a math word problem step by step."""
    problem: str = InputField()
    reasoning: str = OutputField(description="Step-by-step reasoning")
    answer: float = OutputField(description="Numerical answer")

delegate = Delegate(model="claude-3-haiku")

problems = [
    "A Singapore HDB flat costs S$485,000. The buyer pays 25% down and finances the rest at
        2.6% annual interest over 25 years. What is the monthly payment on the loan?",
    "A hawker sells 150 plates of chicken rice per day at S$4.50 each. Operating costs are
        S$280 per day. What is the weekly profit?",
]

# Zero-shot vs CoT comparison
for problem in problems:
    # Zero-shot
    result_zero = delegate.run(MathSolver, problem=problem)
```

```
    print(f"Zero-shot: {result_zero.answer}")

    # CoT (add reasoning instruction)
    result_cot = delegate.run(MathSolver, problem=f"Think step by step. {problem}")
    print(f"CoT: {result_cot.answer}")
    print(f"Reasoning: {result_cot.reasoning}")
```

Try It Yourself

Drill 1. Compare zero-shot, few-shot (3 examples), and CoT prompting on 10 Singapore math word problems. Which technique achieves the highest accuracy? At what cost per problem?

Solution:

```
from kailash_kaizen import LLMCostTracker

tracker = LLMCostTracker(budget_usd=5.00)
delegate = Delegate(model="claude-3-haiku", cost_tracker=tracker)

techniques = {
    "zero_shot": lambda p: p,
    "few_shot": lambda p: f"[examples]\n{p}",
    "cot": lambda p: f"Let's think step by step.\n{p}",
}

for name, transform in techniques.items():
    correct = 0
    for problem, expected in test_problems:
        result = delegate.run(MathSolver, problem=transform(problem))
        if abs(result.answer - expected) < 0.01:
            correct += 1
    print(f"{name}: {correct}/{len(test_problems)} correct, cost=${tracker.total_cost:.4f}")
```

Drill 2. Implement self-consistency: sample 5 CoT responses (temperature=0.7) and take the majority-vote answer. Does self-consistency improve accuracy over single-sample CoT?

Solution:

```
from collections import Counter

def self_consistency(delegate, problem, n_samples=5):
    answers = []
    for _ in range(n_samples):
        result = delegate.run(MathSolver, problem=f"Think step by step.\n{problem}",
                              temperature=0.7)
        answers.append(round(result.answer, 2))
    most_common = Counter(answers).most_common(1)[0][0]
    return most_common
```

Drill 3. Build a classification system using Kaizen Delegate that classifies Singapore news headlines into categories (politics, economy, sports, technology, lifestyle). Use a Signature with typed output. Test on 50 headlines.

Solution:

```
class HeadlineClassifier(Signature):
    """Classify a Singapore news headline into a category."""
    headline: str = InputField()
```

```
    category: str = OutputField(description="One of: politics, economy, sports, technology,
        lifestyle")
    confidence: float = OutputField()
```

Drill 4. Implement cost tracking: process 100 classification requests and report total cost, average cost per request, and cost breakdown by model. Set a budget of S$0.50 and verify the tracker stops when the budget is exhausted.

Solution:

```
tracker = LLMCostTracker(budget_usd=0.50)
delegate = Delegate(model="claude-3-haiku", cost_tracker=tracker)
for headline in headlines[:100]:
    try:
        result = delegate.run(HeadlineClassifier, headline=headline)
    except BudgetExhaustedError:
        print(f"Budget exhausted after {tracker.total_calls} calls")
        break
```

Drill 5. Explain the relationship between temperature, top-p (nucleus sampling), and output quality. Run the same prompt at temperatures 0, 0.3, 0.7, and 1.0 ten times each. Measure the variance of the outputs.

Solution: At temperature 0, all 10 outputs are identical (deterministic). At 0.3, outputs are nearly identical with occasional minor variations. At 0.7, there is meaningful diversity in phrasing but consistent conclusions. At 1.0, outputs vary significantly, including occasional errors. For factual tasks, temperature 0–0.3 is appropriate; for creative tasks, 0.7–1.0.

Cross-References

- **Lesson 5.4** derived the transformer architecture. LLMs are transformers scaled to billions of parameters.
- **Lesson 6.2** will fine-tune LLMs for domain-specific tasks.
- **Lesson 6.3** will align LLMs with human preferences using DPO.
- **Lesson 6.4** will ground LLMs in facts using RAG.

Reflection

You should now be able to:

- Explain how LLMs are pre-trained and aligned.
- Apply five prompt engineering techniques and know when each is appropriate.
- Use Kaizen Delegate for structured, type-safe LLM output.
- Track and budget LLM costs in production.

Lesson 6.2: LLM Fine-Tuning — LoRA, Adapters, and the Technique Landscape

Why This Matters

Prompt engineering is limited. No matter how clever your prompt, the model's knowledge is fixed at pre-training. If you need a model that understands Singapore legal terminology, medical Mandarin-English code-

switching, or your company's internal product taxonomy, you need to fine-tune. But full fine-tuning of a 7-billion-parameter model requires hundreds of gigabytes of GPU memory — impractical for most teams. Parameter-efficient fine-tuning (PEFT) methods like LoRA and adapters achieve comparable results by modifying only a tiny fraction of the parameters.

Core Concepts

THEORY: LoRA — Low-Rank Adaptation

LoRA (Low-Rank Adaptation of Large Language Models) is based on the observation that the weight updates during fine-tuning have low intrinsic rank. Instead of updating the full weight matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$, LoRA decomposes the update into two low-rank matrices:

$$\mathbf{W}' = \mathbf{W}_0 + \mathbf{B}\mathbf{A}$$

where $\mathbf{W}_0$ is the frozen pre-trained weight, $\mathbf{B} \in \mathbb{R}^{d \times r}$ and $\mathbf{A} \in \mathbb{R}^{r \times k}$ are the trainable low-rank matrices, and $r \ll \min(d, k)$ is the rank.

The connection to Module 4: LoRA IS low-rank matrix factorisation (Lesson 4.3, SVD). The pre-trained weights capture the bulk of the model's knowledge; the low-rank update captures the task-specific adaptation. Typical ranks are $r = 4, 8, 16$ — meaning you train a fraction of a percent of the total parameters.

```
import torch
import torch.nn as nn

class LoRALayer(nn.Module):
    def __init__(self, in_features, out_features, rank=8, alpha=16):
        super().__init__()
        self.original = nn.Linear(in_features, out_features, bias=False)
        self.original.weight.requires_grad = False  # freeze original

        self.A = nn.Parameter(torch.randn(in_features, rank) * 0.01)
        self.B = nn.Parameter(torch.zeros(rank, out_features))
        self.scaling = alpha / rank

    def forward(self, x):
        original_output = self.original(x)
        lora_output = x @ self.A @ self.B * self.scaling
        return original_output + lora_output
```

THEORY: Adapter layers

Adapter layers insert small bottleneck modules between transformer layers:

$$\mathbf{h}' = \mathbf{h} + f(\mathbf{h}\mathbf{W}_{\text{down}})\mathbf{W}_{\text{up}}$$

where $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d \times m}$ projects to a lower dimension m, f is an activation function, and $\mathbf{W}_{\text{up}} \in \mathbb{R}^{m \times d}$ projects back. Only $\mathbf{W}_{\text{down}}$ and $\mathbf{W}_{\text{up}}$ are trained; the rest of the model is frozen.

```
class AdapterLayer(nn.Module):
    def __init__(self, d_model, bottleneck=64):
        super().__init__()
        self.down = nn.Linear(d_model, bottleneck)
        self.up = nn.Linear(bottleneck, d_model)
```

```
        self.activation = nn.GELU()

    def forward(self, x):
        return x + self.up(self.activation(self.down(x)))
```

FOUNDATIONS: LoRA vs Adapter comparison

Dimension	LoRA	Adapter
Parameter update	Low-rank matrices A, B	Bottleneck FC layers
Where applied	Weight matrices (Q, K, V, O)	Between transformer layers
Merge at inference	Yes (add B×A to W)	No (module remains)
Inference overhead	Zero	Small (extra forward pass)
Typical parameter %	0.1–1%	1–5%

FOUNDATIONS: The fine-tuning landscape (survey)

Technique	Key Idea	Parameters Trained
Full fine-tuning	Update all weights	100%
LoRA	Low-rank weight update	0.1–1%
Adapters	Bottleneck modules	1–5%
Prefix tuning	Learnable prefix tokens	< 1%
Prompt tuning	Learnable soft prompts	< 0.1%
LLRD	Layer-wise learning rate decay	100% (different LRs)
Progressive freezing	Gradually unfreeze layers	Varies
Knowledge distillation	Teacher-student	100% of student
QLoRA	Quantised base + LoRA	0.1–1%

ADVANCED: Model merging

After fine-tuning multiple LoRA adapters for different tasks, you can merge them:

- **TIES (Trim, Elect Sign, Merge):** trim small values, resolve sign conflicts, merge remaining.
- **DARE (Drop and Rescale):** randomly drop parameters, rescale survivors.
- **SLERP:** spherical linear interpolation between weight vectors.
- **Task arithmetic:** add or subtract fine-tuned weight deltas for compositional control.

FOUNDATIONS: Quantisation

Reduce model precision to fit larger models on smaller hardware:

- **GPTQ:** post-training quantisation using approximate Hessian.
- **AWQ:** activation-aware quantisation that preserves important channels.
- **QLoRA:** quantise the base model to 4-bit, then apply LoRA on top. This allows fine-tuning a 65B model on a single GPU.

Mathematical Foundations

THEORY: Why low rank works

During fine-tuning, the weight update $\Delta\mathbf{W} = \mathbf{W}' - \mathbf{W}_0$ has been empirically observed to have a low intrinsic rank. Aghajanyan et al. (2021) showed that pre-trained models have a low "intrinsic dimensionality" — only a small number of dimensions in parameter space need to change to adapt to a new task. LoRA exploits this by constraining the update to rank r, which acts as a regulariser and reduces the number of trainable parameters from $d \times k$ to $(d + k) \times r$.

For a weight matrix $\mathbf{W} \in \mathbb{R}^{768 \times 768}$ with $r = 8$: full fine-tuning trains $589,824$ parameters; LoRA trains $(768 + 768) \times 8 = 12,288$ parameters — a 48× reduction.

The Kailash Engine: kailash-align

```
from kailash_align import AlignmentPipeline, AlignmentConfig, AdapterRegistry

config = AlignmentConfig(method="lora", rank=8, alpha=16)
pipeline = AlignmentPipeline(config)
pipeline.train(train_dataset, eval_dataset)

# Register the adapter
registry = AdapterRegistry()
registry.register("sentiment_lora", pipeline.adapter)
```

Worked Example: LoRA Fine-Tuning for Sentiment Classification

```
import torch
import torch.nn as nn
from transformers import AutoTokenizer, AutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased",
        num_labels=2)

# Replace attention layers with LoRA versions
for layer in model.bert.encoder.layer:
    original_q = layer.attention.self.query
    layer.attention.self.query = LoRALayer(
        original_q.in_features, original_q.out_features, rank=8
    )
    layer.attention.self.query.original = original_q

# Count parameters
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Total: {total:,}, Trainable: {trainable:,} ({100*trainable/total:.2f}%)")

# Train
optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad],
    lr=1e-4
)
```

Try It Yourself

Drill 1. Implement LoRA from scratch (the `LoRALayer` class) and apply it to a pre-trained BERT model. Fine-tune on IMDB sentiment classification. Report accuracy and compare with full fine-tuning.

Solution:

```
# Apply LoRALayer to Q, K, V projections in all attention layers
# Train for 3 epochs, evaluate on test set
```

Drill 2. Implement adapter layers from scratch and insert them into BERT. Compare adapter fine-tuning with LoRA fine-tuning on the same task: accuracy, parameter count, training time.

Solution:

```
class AdapterBertLayer(nn.Module):
    def __init__(self, original_layer, bottleneck=64):
        super().__init__()
        self.original = original_layer
        self.adapter = AdapterLayer(768, bottleneck)

    def forward(self, *args, **kwargs):
        out = self.original(*args, **kwargs)
        return (self.adapter(out[0]),) + out[1:]
```

Drill 3. Vary the LoRA rank from 1 to 64 (1, 2, 4, 8, 16, 32, 64). Plot accuracy vs rank and training time vs rank. What is the optimal rank for the sentiment task?

Solution:

```
for rank in [1, 2, 4, 8, 16, 32, 64]:
    # Rebuild model with this rank, train, evaluate
    pass
```

Drill 4. Train two LoRA adapters: one for sentiment classification and one for topic classification. Merge them using task arithmetic (add both weight deltas to the base model). Does the merged model perform both tasks?

Solution:

```
# Extract adapter weights: delta_sentiment = LoRA_A @ LoRA_B for sentiment adapter
# Extract adapter weights: delta_topic = LoRA_A @ LoRA_B for topic adapter
# Merged weight = W_0 + delta_sentiment + delta_topic
```

Drill 5. Explain in five sentences how LoRA relates to SVD from Module 4, Lesson 4.3. What is the "low-rank structure" that LoRA exploits? Why does constraining the rank act as a regulariser?

Solution: SVD decomposes a matrix into $\mathbf{U\Sigma V}^T$, where keeping only the top r singular values gives the best rank-r approximation. LoRA's **BA** decomposition is equivalent to constraining the weight update to rank r. The "low-rank structure" is the observation that fine-tuning changes lie in a low-dimensional subspace of the full parameter space. Constraining the rank acts as a regulariser because it limits the model's capacity to overfit to the fine-tuning data — similar to how PCA with fewer components prevents overfitting to noise. This is why LoRA with $r = 8$ often matches full fine-tuning on tasks with moderate training data.

Cross-References

- **Lesson 4.3** derived SVD and PCA. LoRA is low-rank factorisation applied to weight updates.
- **Lesson 5.7** introduced transfer learning with frozen-backbone fine-tuning. LoRA and adapters are parameter-efficient alternatives.
- **Lesson 6.3** will use aligned models produced by fine-tuning.

Reflection

You should now be able to:

- Implement LoRA from scratch and explain the low-rank mathematics.
- Implement adapter layers from scratch.
- Compare all major fine-tuning techniques and select the right one.
- Merge multiple LoRA adapters using task arithmetic or TIES.

Lesson 6.3: Preference Alignment — DPO and GRPO

Why This Matters

A fine-tuned model produces domain-specific outputs, but it may still generate unhelpful, verbose, or harmful responses. Preference alignment trains the model to prefer responses that humans prefer. RLHF (Reinforcement Learning from Human Feedback) was the original approach: train a reward model on human preferences, then use PPO to optimise the LLM against that reward model. But RLHF is complex — it requires training and maintaining a separate reward model, and PPO is notoriously unstable.

DPO (Direct Preference Optimization) achieves the same goal by bypassing the reward model entirely. It derives a closed-form loss function directly from the preference data. GRPO (Group Relative Policy Optimization), used in DeepSeek-R1, takes a different approach: sample multiple completions, score them relative to the group mean, and optimise using policy gradients. Both are simpler and more stable than RLHF.

Core Concepts

THEORY: From RLHF to DPO — the derivation

RLHF maximises the expected reward while staying close to the reference policy:

$$\max_\theta \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot \mid x)}[r(x, y)] - \beta \, \mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$$

The optimal solution to this constrained optimisation is:

$$\pi^*(y \mid x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y \mid x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

Solving for the reward:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x)$$

THEORY: The Bradley-Terry preference model

The Bradley-Terry model defines the probability that response y_w is preferred over y_l given prompt x:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

where σ is the sigmoid function. Substituting the reward expression and noting that $\log Z(x)$ cancels:

$$P(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)$$

THEORY: The DPO loss

The DPO loss maximises the log-likelihood of the observed preferences:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

This is a standard binary classification loss. The model learns to assign higher probability to preferred responses relative to the reference policy. No reward model, no PPO, no RL training loop — just supervised learning on preference pairs.

The hyperparameter β controls how much the aligned model can deviate from the reference policy. Small β: the model stays close to the reference (conservative alignment). Large β: the model can deviate further (aggressive alignment, risk of degradation).

THEORY: GRPO — Group Relative Policy Optimization

GRPO (used in DeepSeek-R1, 2025) takes a different approach:

1. For each prompt x, sample G completions from the current policy.
2. Score each completion using a simple scoring function (not a learned reward model).
3. Compute the advantage relative to the group mean: $\hat{A}_i = r_i - \bar{r}_G$.
4. Update the policy using a clipped policy gradient (similar to PPO).

GRPO shares DPO's advantage of not needing a reward model, but maintains the policy gradient framework. It is particularly effective for reasoning tasks where the scoring function can be binary (correct/incorrect) and multiple samples provide a natural relative ranking.

FOUNDATIONS: LLM-as-Judge

Use one LLM to evaluate another's outputs. The judge LLM rates responses on criteria like helpfulness, factual accuracy, and harmlessness. Known biases:

- **Position bias:** the judge prefers the response that appears first.
- **Verbosity bias:** the judge prefers longer responses.
- **Self-enhancement bias:** the judge prefers responses similar to its own style.

Mitigations: swap response positions and average, normalise by length, use multiple judge models.

FOUNDATIONS: Evaluation benchmarks

Benchmark	Tests	Format
MMLU	Multi-task language understanding	Multiple choice
HellaSwag	Commonsense reasoning	Sentence completion
HumanEval	Code generation	Function implementation
MT-Bench	Multi-turn conversation	Open-ended + judge

The Kailash Engine: kailash-align (DPO)

```
from kailash_align import AlignmentPipeline, AlignmentConfig

config = AlignmentConfig(method="dpo", beta=0.1, epochs=3)
pipeline = AlignmentPipeline(config)
pipeline.train(preference_dataset)
```

Worked Example: DPO Training on Preference Data

```
import torch
import torch.nn.functional as F

def dpo_loss(model, ref_model, chosen_ids, rejected_ids, beta=0.1):
    # Forward pass for chosen and rejected
    chosen_logps = get_log_probs(model, chosen_ids)
    rejected_logps = get_log_probs(model, rejected_ids)

    with torch.no_grad():
        ref_chosen_logps = get_log_probs(ref_model, chosen_ids)
        ref_rejected_logps = get_log_probs(ref_model, rejected_ids)

    # DPO loss
    chosen_rewards = beta * (chosen_logps - ref_chosen_logps)
    rejected_rewards = beta * (rejected_logps - ref_rejected_logps)
    loss = -F.logsigmoid(chosen_rewards - rejected_rewards).mean()
    return loss

def get_log_probs(model, input_ids):
    logits = model(input_ids).logits
    log_probs = F.log_softmax(logits, dim=-1)
    # Sum log probs of actual tokens
    token_log_probs = log_probs.gather(-1, input_ids[:, 1:].unsqueeze(-1)).squeeze(-1)
    return token_log_probs.sum(dim=-1)
```

Try It Yourself

Drill 1. Implement the DPO loss function from scratch. Verify it decreases during training on a small preference dataset.

Solution:

```
# Create synthetic preference pairs
# Train for 10 epochs, plot loss curve
```

Drill 2. Vary β from 0.01 to 1.0 ($0.01, 0.05, 0.1, 0.5, 1.0$). How does β affect the trade-off between alignment and generation quality? Report win rate against the reference model for each β.

Solution:

```
for beta in [0.01, 0.05, 0.1, 0.5, 1.0]:
    # Train DPO with this beta
    # Evaluate win rate using LLM-as-judge
    pass
```

Drill 3. Implement LLM-as-judge evaluation. Measure position bias by evaluating the same pair of responses in both orderings. Report the bias magnitude.

Solution:

```
class JudgeSignature(Signature):
    """Judge which response is better."""
    prompt: str = InputField()
    response_a: str = InputField()
    response_b: str = InputField()
    winner: str = OutputField(description="A or B")

# Run with AB order and BA order, measure disagreement rate
```

Drill 4. Compare DPO with supervised fine-tuning (SFT) on the same dataset. SFT trains only on the preferred responses; DPO trains on both preferred and dispreferred. Which produces better alignment? Why does contrastive learning (DPO) help?

Solution: DPO outperforms SFT because it learns from both positive and negative examples. SFT only teaches the model what to produce; DPO also teaches what to avoid. The contrastive signal is more informative.

Drill 5. Explain GRPO in three sentences. When would you choose GRPO over DPO? When would you choose DPO over GRPO?

Solution: GRPO samples multiple completions per prompt, scores them relative to the group mean, and optimises using policy gradients. Choose GRPO when you have a cheap, reliable scoring function (e.g., code correctness, math verification) that can evaluate completions without human annotation. Choose DPO when you have a fixed dataset of human preferences and want a simpler, offline training procedure.

Cross-References

- **Lesson 5.8** introduced PPO for reinforcement learning. RLHF uses PPO to align LLMs; DPO bypasses PPO.
- **Lesson 6.1** introduced LLM fundamentals and alignment overview.
- **Lesson 6.7** will use PACT governance to enforce alignment boundaries beyond what DPO can learn.

Reflection

You should now be able to:

- Derive DPO from the RLHF objective via the Bradley-Terry model.
- Implement DPO training from scratch.
- Explain the role of β in controlling alignment strength.
- Evaluate aligned models using LLM-as-judge and standard benchmarks.
- Compare DPO, GRPO, and RLHF and know when each is appropriate.

Lesson 6.4: RAG Systems

Why This Matters

LLMs have a knowledge cutoff — they do not know about events after their training data ends. They hallucinate — they generate confident, plausible-sounding text that is factually wrong. RAG (Retrieval-Augmented Generation) solves both problems by grounding LLM responses in retrieved documents. Instead of relying on parametric memory (what the model learned during training), RAG uses non-parametric memory (a searchable document store) to provide relevant context.

Core Concepts

FOUNDATIONS: The RAG pipeline

1. **Chunk** documents into manageable pieces (paragraphs, sentences, or semantic units).
2. **Embed** each chunk using a sentence embedding model, producing dense vectors.
3. **Index** the vectors in a vector database for fast nearest-neighbour search.
4. **Retrieve** relevant chunks given a query (dense, sparse, or hybrid retrieval).
5. **Generate** a response using the retrieved chunks as context.

THEORY: BM25 — sparse retrieval

BM25 (from Lesson 4.6) scores documents using term frequency with saturation and document-length normalisation:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{f_{t,d} (k_1 + 1)}{f_{t,d} + k_1 (1 - b + b \cdot |d|/|d_{\text{avg}}|)}$$

BM25 is fast, interpretable, and excels at exact keyword matching. It fails on semantic queries ("What are the rules for HDB ownership?" will not match a document about "public housing eligibility criteria" unless the exact words overlap).

THEORY: Cosine similarity — dense retrieval

Dense retrieval embeds both the query and documents as dense vectors, then finds the most similar:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$$

Dense retrieval captures semantic similarity ("HDB ownership" matches "public housing eligibility") but can miss exact terms.

FOUNDATIONS: Hybrid retrieval

Combine BM25 and dense retrieval using reciprocal rank fusion:

$$\text{RRF}(d) = \sum_{r \in \text{rankers}} \frac{1}{k + \text{rank}_r(d)}$$

where k is a constant (typically 60). This captures both exact keyword matches and semantic similarity.

FOUNDATIONS: RAGAS evaluation

RAGAS provides four metrics for RAG quality:

- **Faithfulness:** is the answer supported by the retrieved context?
- **Answer relevance:** does the answer address the question?
- **Context relevance:** are the retrieved chunks relevant to the question?
- **Context recall:** did the retrieval find all relevant information?

ADVANCED: HyDE (Hypothetical Document Embeddings)

Generate a hypothetical answer to the query (even if wrong), embed it, and use that embedding for retrieval. The intuition: the hypothetical answer is more semantically similar to the actual relevant documents than the short

query is.

Worked Example: RAG on Singapore Policy Documents

```
from sentence_transformers import SentenceTransformer
import numpy as np

# 1. Chunk documents
def chunk_text(text, chunk_size=500, overlap=100):
    words = text.split()
    chunks = []
    for i in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[i:i + chunk_size])
        chunks.append(chunk)
    return chunks

# 2. Embed chunks
embedder = SentenceTransformer("all-MiniLM-L6-v2")
chunk_embeddings = embedder.encode(chunks)

# 3. Retrieve
def retrieve(query, top_k=5):
    query_embedding = embedder.encode([query])
    similarities = np.dot(chunk_embeddings, query_embedding.T).flatten()
    top_indices = np.argsort(similarities)[-top_k:][::-1]
    return [chunks[i] for i in top_indices]

# 4. Generate
context = "\n".join(retrieve("What are the HDB eligibility criteria?"))
prompt = f"Based on the following context, answer the question.\n\nContext:
        {context}\n\nQuestion: What are the HDB eligibility criteria?"
```

Try It Yourself

Drill 1. Implement BM25 retrieval from scratch. Compare it with dense retrieval on 20 Singapore policy questions. Which performs better on keyword-heavy queries? On semantic queries?

Drill 2. Implement hybrid retrieval using reciprocal rank fusion. Does it outperform both BM25 and dense retrieval individually?

Drill 3. Evaluate your RAG system using RAGAS. Compute faithfulness and context relevance for 10 questions.

Drill 4. Implement HyDE. Compare retrieval quality (recall@5) with and without HyDE.

Drill 5. Vary the chunk size (100, 250, 500, 1000 words) and overlap (0%, 20%, 50%). How do these parameters affect retrieval quality and answer quality?

Cross-References

- **Lesson 4.6** introduced TF-IDF and BM25. BM25 is the sparse retrieval backbone of RAG.
- **Lesson 6.1** covered prompt engineering. RAG is prompt engineering with dynamic context.
- **Lesson 6.5** will use RAG as a tool within agents.

Reflection

You should now be able to build a complete RAG pipeline, compare retrieval methods, and evaluate with RAGAS.

Lesson 6.5: AI Agents — ReAct, Tool Use, and Function Calling

Why This Matters

An LLM generates text. An agent generates actions. The difference is that an agent can observe the results of its actions and adjust its behaviour accordingly. A ReAct agent follows a thought-action-observation loop: it reasons about the task, takes an action (call a tool, search a database, run code), observes the result, and decides what to do next.

Core Concepts

THEORY: ReAct formalisation

ReAct (Reasoning + Acting) interleaves reasoning traces with actions:

$$\text{Thought}_t \to \text{Action}_t \to \text{Observation}_t \to \text{Thought}_{t+1} \to \dots$$

The thought is free-form text where the agent reasons about the current state. The action invokes a tool with specific parameters. The observation is the tool's output. The loop continues until the agent produces a final answer.

FOUNDATIONS: Function calling

Function calling provides structured tool invocation:

```
tools = [
    {
        "name": "search_database",
        "description": "Search the Singapore property database",
        "parameters": {
            "type": "object",
            "properties": {
                "town": {"type": "string"},
                "min_price": {"type": "number"},
                "max_price": {"type": "number"},
            },
        },
    }
]
```

The LLM selects the appropriate tool and fills in the parameters based on the user's natural language query. This bridges natural language understanding with structured API calls.

FOUNDATIONS: Cost budget safety

Agents can enter infinite loops or make expensive API calls. The `LLMCostTracker` enforces a hard budget:

```
from kailash_kaizen import ReActAgent, LLMCostTracker

tracker = LLMCostTracker(budget_usd=2.00)
agent = ReActAgent(
    tools=[data_explorer_tool, training_tool, viz_tool],
    cost_tracker=tracker,
)
result = agent.run("Analyse the HDB dataset and build a price prediction model")
```

FOUNDATIONS: Agent design framework

From the deck: when designing an agent, ask four questions:

1. **What is our goal?** — the task in concrete terms.
2. **What is our thought process?** — the reasoning steps.
3. **What kind of specialist would we hire?** — be precise ("ML data analyst" not "researcher").
4. **What tools do they need?** — versatile, fault-tolerant, with caching.

Worked Example: Data Analysis Agent with ReAct

```
from kailash_kaizen import ReActAgent, Tool
from kailash_ml import DataExplorer, TrainingPipeline, ModelVisualizer

@Tool(description="Profile a dataset and return summary statistics")
def profile_data(dataset_name: str) -> str:
    df = loader.load("mlfp06", dataset_name)
    explorer = DataExplorer()
    profile = explorer.profile(df)
    return str(profile.summary)

@Tool(description="Train a model on the dataset")
def train_model(dataset_name: str, target: str, model_type: str = "xgboost") -> str:
    df = loader.load("mlfp06", dataset_name)
    pipeline = TrainingPipeline(model_type=model_type)
    result = pipeline.train(df, target=target)
    return f"Accuracy: {result.metrics['accuracy']:.3f}"

agent = ReActAgent(tools=[profile_data, train_model])
result = agent.run("Analyse sg_hdb_prices.csv and predict resale_price")
```

Try It Yourself

Drill 1. Build a ReAct agent with three tools (data profiler, model trainer, visualiser). Test it on an end-to-end analysis task.

Drill 2. Implement a cost budget that stops the agent after S$1.00. What happens when the budget is exhausted mid-task?

Drill 3. Build a function-calling agent with structured tool schemas. Compare with the ReAct agent on the same task.

Drill 4. Add error handling: if a tool call fails, the agent should retry with different parameters or use an alternative tool.

Drill 5. Implement an iterative refinement pattern: after the initial analysis, a "critic" agent evaluates the result and suggests improvements, then the original agent implements them.

Cross-References

- **Lesson 6.1** introduced Kaizen Delegate. Agents extend delegates with tool use and reasoning loops.
- **Lesson 6.6** will orchestrate multiple agents.
- **Lesson 6.7** will govern agents with PACT.

Reflection

You should now be able to build ReAct agents with custom tools, implement function calling, and enforce cost budgets.

Lesson 6.6: Multi-Agent Orchestration and MCP

Why This Matters

Complex tasks require multiple specialists. A data analysis task might need a data profiler, a feature engineer, a model trainer, and a report writer — each with different tools and expertise. Multi-agent orchestration coordinates these specialists, and MCP (Model Context Protocol) provides the standard for exposing tools to agents at scale.

Core Concepts

FOUNDATIONS: Multi-agent patterns

Supervisor-worker. One supervisor agent delegates sub-tasks to specialist workers. The supervisor decides which worker to call and aggregates results.

Sequential. Output of one agent feeds into the next: DataScientist -> FeatureEngineer -> ModelSelector -> ReportWriter.

Parallel. Multiple agents work simultaneously on independent sub-tasks. Results are aggregated.

Handoff. An agent transfers control to a specialist when it detects a topic outside its expertise.

FOUNDATIONS: MCP (Model Context Protocol)

MCP standardises how tools are exposed to agents:

```
from kailash_mcp import MCPServer, Tool

server = MCPServer("ml-tools")

@server.tool(description="Explore a dataset")
async def explore(dataset_name: str) -> dict:
    df = loader.load("mlfp06", dataset_name)
    return {"rows": df.height, "columns": df.width}

@server.tool(description="Train a model")
async def train(dataset: str, target: str) -> dict:
    pipeline = TrainingPipeline()
```

```
    result = pipeline.train(loader.load("mlfp06", dataset), target=target)
    return result.metrics

server.run(transport="stdio")
```

FOUNDATIONS: Agent memory

- **Short-term memory:** the current conversation context.
- **Long-term memory:** persistent knowledge across sessions (stored in a database or file).
- **Entity memory:** structured knowledge about people, places, and concepts.

Worked Example: Multi-Agent ML Pipeline

```
from kailash_kaizen import BaseAgent, Delegate

class DataScientistAgent(BaseAgent):
    """Explores data and identifies modelling opportunities."""
    tools = [profile_data, visualise_data]

class FeatureEngineerAgent(BaseAgent):
    """Engineers features from raw data."""
    tools = [create_features, validate_features]

class ModelSelectorAgent(BaseAgent):
    """Selects and trains the best model."""
    tools = [train_model, evaluate_model]

class ReportWriterAgent(BaseAgent):
    """Writes a human-readable report."""
    tools = [generate_report]

# Sequential orchestration
data_analysis = DataScientistAgent().run("Analyse sg_hdb_prices.csv")
features = FeatureEngineerAgent().run(f"Engineer features based on: {data_analysis}")
model = ModelSelectorAgent().run(f"Train model with features: {features}")
report = ReportWriterAgent().run(f"Write report for: {model}")
```

Try It Yourself

Drill 1. Implement the full 4-agent sequential pipeline. Verify that each agent's output is consumed by the next.

Drill 2. Build an MCP server exposing three ML tools. Connect an agent to the MCP server and verify it can discover and use the tools.

Drill 3. Implement supervisor-worker orchestration. The supervisor receives a complex task, breaks it into sub-tasks, delegates each to a specialist, and aggregates the results.

Drill 4. Add long-term memory to an agent using a simple JSON file store. The agent should remember insights from previous sessions.

Drill 5. Implement parallel execution: launch two agents simultaneously (data profiling and feature engineering) and aggregate their results.

Cross-References

- **Lesson 6.5** built single agents. This lesson coordinates multiple agents.
- **Lesson 6.7** will govern multi-agent systems with PACT.

Reflection

You should now be able to implement multi-agent patterns, build MCP servers, and configure agent memory.

Lesson 6.7: AI Governance Engineering

Why This Matters

An ungoverned AI agent is a liability. It can access data it should not see, spend more money than budgeted, take actions outside its intended scope, and produce no audit trail of its decisions. PACT (Policy, Access, Controls, Trust) is the Kailash governance framework that turns these risks into engineering constraints. Governance is not philosophy — it is code. Access controls you implement, operating envelopes you define, and budget cascading you test.

Core Concepts

THEORY: PACT D/T/R addressing

PACT structures access control using a three-part address:

- **D (Domain):** the area of knowledge or operation (e.g., “finance”, “healthcare”, “ml-ops”).
- **T (Team):** the organisational unit (e.g., “data-science”, “compliance”, “engineering”).
- **R (Role):** the specific role within the team (e.g., “analyst”, “auditor”, “admin”).

Access decisions are made by checking whether the requester’s D/T/R address has permission for the requested resource.

```
from kailash_pact import GovernanceEngine, Address

engine = GovernanceEngine()
org = engine.compile_org({
    "domains": ["finance", "marketing"],
    "teams": ["data-science", "compliance"],
    "roles": ["analyst", "auditor", "admin"],
})

requester = Address(domain="finance", team="data-science", role="analyst")
can_access = engine.can_access(requester, resource="customer_data")
explanation = engine.explain_access(requester, resource="customer_data")
```

FOUNDATIONS: Operating envelopes

An operating envelope defines the boundaries of what an agent can do:

- **Task envelope:** restricts the agent to specific task types (e.g., “data analysis only, no model deployment”).
- **Role envelope:** restricts based on organisational role (e.g., “read-only for analysts, read-write for engineers”).

- **Monotonic tightening:** envelopes can only get stricter, never looser. An agent cannot grant itself additional permissions.

FOUNDATIONS: Budget cascading

Parent agents allocate budgets to child agents:

```
from kailash_kaizen import LLMCostTracker

parent_tracker = LLMCostTracker(budget_usd=10.00)
child_1_tracker = parent_tracker.allocate(budget_usd=3.00)
child_2_tracker = parent_tracker.allocate(budget_usd=3.00)
# Parent retains $4.00 for its own use
# When child_1 exhausts $3.00, it stops — does not consume parent's remaining budget
```

FOUNDATIONS: Governance testing

Test that governance works — denied access must stay denied:

```
def test_analyst_cannot_delete():
    analyst = Address(domain="finance", team="data-science", role="analyst")
    assert not engine.can_access(analyst, resource="customer_data", action="delete")

def test_admin_can_delete():
    admin = Address(domain="finance", team="data-science", role="admin")
    assert engine.can_access(admin, resource="customer_data", action="delete")

def test_envelope_cannot_loosen():
    agent = PactGovernedAgent(envelope={"tasks": ["analysis"]})
    with pytest.raises(EnvelopeViolation):
        agent.envelope.add_task("deployment")  # monotonic tightening
```

FOUNDATIONS: PactGovernedAgent

```
from kailash_pact import PactGovernedAgent

agent = PactGovernedAgent(
    address=Address(domain="finance", team="data-science", role="analyst"),
    envelope={"tasks": ["analysis", "reporting"], "max_cost_usd": 5.00},
    enforcement="block",  # or "warn" or "audit"
)
# The agent cannot invoke tools outside its envelope
# All access decisions are logged for audit
```

Worked Example: Governed Multi-Agent System

```
from kailash_pact import GovernanceEngine, PactGovernedAgent, Address

engine = GovernanceEngine()
org = engine.compile_org({
    "domains": ["retail"],
    "teams": ["analytics", "operations"],
    "roles": ["analyst", "manager", "admin"],
})

# Analyst agent: can profile and visualise, cannot train or deploy
analyst_agent = PactGovernedAgent(
```

```
    address=Address(domain="retail", team="analytics", role="analyst"),
    envelope={"tasks": ["profile", "visualise"]},
    cost_budget=2.00,
)

# Manager agent: can also train models
manager_agent = PactGovernedAgent(
    address=Address(domain="retail", team="analytics", role="manager"),
    envelope={"tasks": ["profile", "visualise", "train"]},
    cost_budget=5.00,
)

# Test governance
assert not analyst_agent.can_perform("train")   # blocked
assert manager_agent.can_perform("train")        # allowed

# Audit trail
print(engine.audit_log())  # all access decisions logged
```

Try It Yourself

Drill 1. Implement a D/T/R access control system. Define 3 domains, 3 teams, and 3 roles. Create 5 access rules and test all boundary cases.

Drill 2. Implement operating envelopes for two agents. Verify that the analyst agent is blocked from training models and the manager agent is allowed.

Drill 3. Implement budget cascading: a supervisor with S$10 allocates S$3 to each of 3 workers. Verify that each worker stops at its budget limit without consuming the supervisor's remaining budget.

Drill 4. Write governance tests that verify: (a) denied access stays denied, (b) envelopes cannot be loosened, (c) budget allocation is respected.

Drill 5. Generate an audit trail for a multi-agent workflow. The trail should log every access decision (who, what, when, allowed/denied, reason).

Cross-References

- **Lesson 6.5** and **6.6** built agents without governance. This lesson adds the safety layer.
- **Lesson 6.8** will deploy governed agents to production.
- **Module 3, Lesson 3.6** covered fairness and model cards. PACT governance is the production enforcement of those principles.

Reflection

You should now be able to:

- Implement PACT governance with D/T/R addressing.
- Define and enforce operating envelopes for agents.
- Implement budget cascading across agent hierarchies.
- Test that governance rules are enforced (denied access stays denied).
- Generate audit trails for compliance.

Lesson 6.8: Capstone — Full Production Platform

Why This Matters

This is the last lesson of the MLFP programme. Everything you have learned converges here: data pipelines from Module 1, statistics from Module 2, the ML pipeline from Module 3, unsupervised learning from Module 4, deep learning architectures from Module 5, and LLMs, agents, and governance from Module 6.

In this lesson you will deploy a complete, governed AI system using Nexus — Kailash's multi-channel deployment platform. The system will be accessible via API, CLI, and MCP simultaneously. It will have authentication, drift monitoring, and governance enforcement in production. This is not a toy — it is the architecture of a real production AI application.

Core Concepts

FOUNDATIONS: Nexus multi-channel deployment

Nexus deploys a single codebase to three interfaces simultaneously:

- **API:** REST endpoints for programmatic access.
- **CLI:** command-line interface for operators.
- **MCP:** Model Context Protocol for AI agent access.

```
from kailash_nexus import Nexus

app = Nexus("ml-platform")

@app.route("/predict")
async def predict(request):
    return model.predict(request.data)

@app.cli_command("predict")
async def predict_cli(data_path: str):
    df = pl.read_csv(data_path)
    return model.predict(df)

app.expose_mcp()  # automatically expose routes as MCP tools
app.run(port=8000)
```

FOUNDATIONS: Authentication and authorisation

```
from kailash_nexus import auth

@app.route("/predict", auth=auth.require_role("analyst"))
async def predict(request):
    return model.predict(request.data)

@app.route("/deploy", auth=auth.require_role("admin"))
async def deploy(request):
    registry.promote(request.model_id, stage="production")
```

FOUNDATIONS: Full platform integration

The complete stack:

1. **Train** a model with `TrainingPipeline`.
2. **Persist** to database with `DataFlow`.
3. **Wrap** in an agent with `Kaizen`.
4. **Govern** the agent with `PACT`.
5. **Deploy** with `Nexus`.
6. **Monitor** with `DriftMonitor`.

FOUNDATIONS: Debugging agent reasoning

When a multi-agent system produces unexpected output, you need to trace the reasoning chain:

```
result = agent.run("Analyse sales data", return_trace=True)
for step in result.trace:
    print(f"Thought: {step.thought}")
    print(f"Action: {step.action}")
    print(f"Observation: {step.observation}")
```

Worked Example: Deploying the M6 System

```
from kailash_nexus import Nexus
from kailash_pact import GovernanceEngine, PactGovernedAgent
from kailash_ml import DriftMonitor, ModelRegistry

# 1. Load the trained model
registry = ModelRegistry()
model = registry.load("hdb_predictor", stage="production")

# 2. Set up governance
engine = GovernanceEngine()
governed_agent = PactGovernedAgent(
    address=Address(domain="property", team="analytics", role="analyst"),
    envelope={"tasks": ["predict", "explain"]},
    cost_budget=5.00,
)

# 3. Set up drift monitoring
monitor = DriftMonitor(reference_data=training_data)

# 4. Deploy with Nexus
app = Nexus("hdb-predictor")

@app.route("/predict", auth=auth.require_role("analyst"))
async def predict(request):
    # Check governance
    if not governed_agent.can_perform("predict"):
        return {"error": "Access denied"}, 403

    # Make prediction
    prediction = model.predict(request.data)

    # Monitor for drift
    drift_report = monitor.check(request.data)
    if drift_report.psi > 0.25:
        print(f"WARNING: Major drift detected (PSI={drift_report.psi:.3f})")

    return {"prediction": prediction, "drift_status": drift_report.status}

@app.route("/health")
```

```
async def health():
    return {"status": "healthy", "model_version": model.version}

app.expose_mcp()
app.run(port=8000)
```

Try It Yourself

Drill 1. Deploy the HDB predictor via Nexus. Test it via all three channels: API (curl/httpx), CLI, and MCP.

Drill 2. Add RBAC authentication. Verify that unauthenticated requests are rejected with 401 and unauthorised requests with 403.

Drill 3. Integrate DriftMonitor. Send data that triggers a drift warning (e.g., data from a different time period). Verify the warning is logged.

Drill 4. Add governance enforcement at the Nexus level. Verify that an analyst can predict but cannot deploy, and an admin can do both.

Drill 5. Generate a complete audit trail for a request that flows through: authentication -> governance check -> prediction -> drift monitoring -> response. Every step should be logged.

Cross-References

- **Module 3, Lesson 3.8** introduced drift monitoring with PSI and KS tests. This lesson deploys it in production.
- **Lesson 6.7** defined governance rules. This lesson enforces them at the API level.
- **Lesson 6.6** built MCP servers. This lesson exposes them through Nexus.

Reflection

You should now be able to:

- Deploy a complete AI system with Nexus (API + CLI + MCP).
- Implement authentication and authorisation.
- Integrate drift monitoring in production.
- Enforce governance at the deployment level.
- Debug agent reasoning chains.

This is the end of the MLFP programme. You started in Module 1 not knowing what a variable was. You are ending Module 6 having deployed a governed, multi-channel AI system that trains models, aligns them with human preferences, grounds them in retrieved knowledge, coordinates multiple specialist agents, enforces access controls, monitors for drift, and serves predictions via API, CLI, and MCP simultaneously.

The distance you have covered is not measured in modules or lines of code. It is measured in the questions you can now ask — and answer. When someone shows you a model, you ask: how was it trained? When they show you a prediction, you ask: how confident is it, and has the input distribution shifted? When they show you an agent, you ask: what are its operating envelopes, and who audits its decisions?

Those are the questions of an ML engineer. Welcome to the profession.

Chapter Summary

Module 6 covered the complete journey from a trained model to a governed production system:

Lesson	Topic	Key Concept
6.1	LLM Fundamentals	Prompt engineering, structured output
6.2	Fine-Tuning	LoRA, adapters, PEFT landscape
6.3	Alignment	DPO, GRPO, Bradley-Terry
6.4	RAG	Retrieval-augmented generation
6.5	Agents	ReAct, tool use, cost budgets
6.6	Multi-Agent	Orchestration, MCP, memory
6.7	Governance	PACT D/T/R, operating envelopes
6.8	Capstone	Nexus deployment, full integration

The progression mirrors the real-world deployment pipeline: understand the model (6.1), customise it (6.2), align it (6.3), ground it (6.4), give it tools (6.5), coordinate specialists (6.6), govern the system (6.7), and ship it (6.8).

The complete MLFP arc

Module	Theme	Key Skill
M1	Data foundations	Polars, visualisation, profiling
M2	Statistics and probability	Inference, hypothesis testing, regression
M3	Supervised ML	Full pipeline, evaluation, deployment
M4	Unsupervised ML + Neural bridge	Clustering, PCA, embeddings, backpropagation
M5	Deep learning architectures	CNN, RNN, Transformer, GAN, GNN, RL
M6	LLMs and production systems	Fine-tuning, RAG, agents, governance, deployment

You have traversed the Feature Engineering Spectrum from manual features to learned features to semantic features. You have built models from linear regression to transformers to governed multi-agent systems. You have deployed with Kailash's full stack: Core SDK, DataFlow, ML, Kaizen, PACT, Nexus, and Align.

The programme is complete. The learning continues.

Glossary

Adapter layer. A small bottleneck module inserted between transformer layers for parameter-efficient fine-tuning.

Agent. A system that reasons about tasks, takes actions, observes results, and iterates. Distinct from a model, which simply maps inputs to outputs.

Alignment. Training a model to produce outputs that are helpful, harmless, and honest, typically through RLHF or DPO.

BM25. A sparse retrieval scoring function that extends TF-IDF with term frequency saturation and document length normalisation.

Bradley-Terry model. A probabilistic model for pairwise preferences: $P(y_w > y_l) = \sigma(r(y_w) - r(y_l))$.

Budget cascading. Allocating cost budgets from parent agents to child agents, ensuring each operates within its allocation.

Chain-of-thought (CoT). A prompting technique that instructs the model to reason step by step before answering.

Chunking. Splitting documents into smaller pieces for retrieval in a RAG system.

Cosine similarity. A similarity measure between vectors based on the cosine of the angle between them.

D/T/R addressing. PACT's three-part access control structure: Domain, Team, Role.

Delegate. Kaizen's API for structured LLM interaction with cost tracking.

Dense retrieval. Finding similar documents using vector embeddings and cosine similarity.

DPO (Direct Preference Optimization). An alignment method that bypasses the reward model by deriving a loss function directly from preference data.

Enforcement mode. How PACT handles governance violations: warn (log but allow), block (deny), or audit (log for review).

Fail-closed. A governance policy where access is denied by default if the governance check fails or is unavailable.

Few-shot prompting. Providing examples in the prompt to guide the model's output format and quality.

Function calling. Structured tool invocation where the LLM selects a tool and fills in parameters based on natural language input.

GovernanceEngine. PACT's core component that compiles organisational structures and evaluates access requests.

GRPO (Group Relative Policy Optimization). An alignment method that scores multiple completions relative to the group mean, used in DeepSeek-R1.

HyDE (Hypothetical Document Embeddings). A RAG technique that generates a hypothetical answer and uses its embedding for retrieval.

InferenceServer. Kailash ML engine for serving model predictions in production.

KV-cache. Stored key and value matrices from previous attention computations, avoiding redundant calculation during autoregressive generation.

LLM-as-Judge. Using one LLM to evaluate another's outputs on criteria like helpfulness and accuracy.

LoRA (Low-Rank Adaptation). A parameter-efficient fine-tuning method that decomposes weight updates into low-rank matrices.

MCP (Model Context Protocol). A protocol for exposing tools to AI agents at scale with standardised schemas.

Model merging. Combining multiple fine-tuned model weights (TIES, DARE, SLERP, task arithmetic).

Monotonic tightening. The principle that operating envelopes can only become stricter, never looser.

Nexus. Kailash's multi-channel deployment platform (API + CLI + MCP simultaneously).

Operating envelope. Defined boundaries for what an agent can do, enforced by PACT.

PACT. Policy, Access, Controls, Trust. Kailash's governance framework for AI systems.

PactGovernedAgent. An agent wrapper that enforces PACT governance rules.

Preference alignment. Training a model to prefer outputs that humans prefer, using methods like DPO or RLHF.

Prompt engineering. Designing input prompts to control LLM output quality and format.

QLoRA. Quantising the base model to 4-bit precision, then applying LoRA for fine-tuning.

Quantisation. Reducing model weight precision (e.g., from FP16 to INT4) to reduce memory and compute requirements.

RAG (Retrieval-Augmented Generation). Grounding LLM responses in retrieved documents to reduce hallucination and incorporate current knowledge.

RAGAS. An evaluation framework for RAG systems measuring faithfulness, relevance, and recall.

RBAC (Role-Based Access Control). Granting permissions based on the user's role in the organisation.

ReAct. A reasoning framework where agents interleave thought, action, and observation steps.

Reference policy. The original, unaligned model used as a baseline in DPO to prevent the aligned model from deviating too far.

Self-consistency. Sampling multiple reasoning paths and taking the majority vote to reduce variance.

Signature. Kaizen's typed input/output schema for structured LLM interaction.

Sparse retrieval. Finding documents using keyword matching (BM25, TF-IDF).

Speculative decoding. Using a small model to draft tokens that a larger model verifies, accelerating generation.

Structured output. LLM output parsed into typed fields rather than free-form text.

Temperature. A parameter controlling the randomness of LLM output. Low temperature = deterministic; high = random.

Tool use. An agent's ability to invoke external functions (APIs, databases, code execution) based on reasoning.

Zero-shot prompting. Providing only a task description with no examples.

Further Reading

On LLMs and prompt engineering

- Brown, T., et al. "Language Models are Few-Shot Learners." *NeurIPS*, 2020. The GPT-3 paper introducing few-shot prompting.
- Wei, J., et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *NeurIPS*, 2022.
- Wang, X., et al. "Self-Consistency Improves Chain of Thought Reasoning." *ICLR*, 2023.

On fine-tuning

- Hu, E., et al. "LoRA: Low-Rank Adaptation of Large Language Models." *ICLR*, 2022. The original LoRA paper.
- Houlsby, N., et al. "Parameter-Efficient Transfer Learning for NLP." *ICML*, 2019. The adapter layers paper.
- Aghajanyan, A., Gupta, S., and Zettlemoyer, L. "Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning." *ACL*, 2021.
- Dettmers, T., et al. "QLoRA: Efficient Finetuning of Quantized Language Models." *NeurIPS*, 2023.

On preference alignment

- Rafailov, R., et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." *NeurIPS*, 2023. The DPO paper.
- Shao, Z., et al. "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models." *arXiv:2402.03300*, 2024. Introduces GRPO.
- Ouyang, L., et al. "Training language models to follow instructions with human feedback." *NeurIPS*, 2022. The InstructGPT/RLHF paper.

On RAG

- Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS*, 2020. The original RAG paper.
- Gao, L., et al. "Precise Zero-Shot Dense Retrieval without Relevance Labels." *ACL*, 2023. The HyDE paper.

On AI agents

- Yao, S., et al. "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR*, 2023. The ReAct paper.
- Schick, T., et al. "Toolformer: Language Models Can Teach Themselves to Use Tools." *NeurIPS*, 2023.

On AI governance

- The Terrene Foundation. *PACT: Policy, Access, Controls, Trust — A Governance Framework for AI Agent Organizations*. 2024.
- Mitchell, M., et al. "Model Cards for Model Reporting." *FAT**, 2019.

On production ML systems

- Sculley, D., et al. "Hidden Technical Debt in Machine Learning Systems." *NeurIPS*, 2015. The classic paper on ML system complexity.
- Paleyes, A., Urma, R.-G., and Lawrence, N. "Challenges in Deploying Machine Learning: A Survey of Case Studies." *ACM Computing Surveys*, 2022.